

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Оглавление

Форматирование строк	2
Оператор %	2
Метод str.format()	4
f-строки	6
Форматирование чисел.....	9

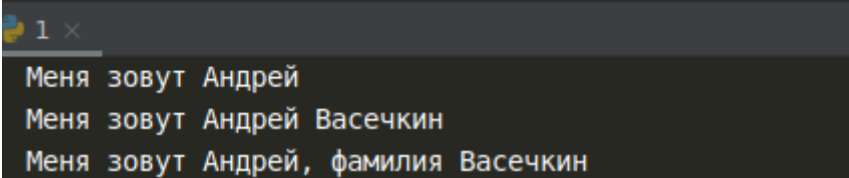
Форматирование строк

Форматирование необходимо, когда возникают ситуации, если нужно сделать строку, подставив в неё некоторые данные, полученные в процессе выполнения программы (пользовательский ввод, данные из файлов и т. д.). Подстановку данных можно сделать с помощью различных вариантов.

Оператор %

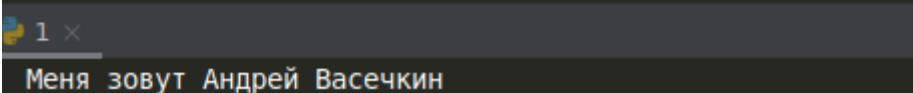
Работа со строчным типом данных указывается как `%s`, вместо этой комбинации будет подставляться переменная, указанная после оператора `%`. Если переменных несколько, то они указываются в круглых скобках в порядке очереди через запятую.

```
name = 'Андрей'
surname = 'Васечкин'
print("Меня зовут %s" % name)
print("Меня зовут %s %s" % (name, surname))
print("Меня зовут %s, фамилия %s" % (name, surname))
```



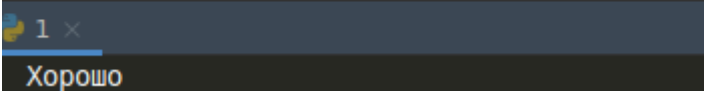
Также можно не использовать переменные, а подставлять данные сразу в качестве аргументов в скобки.

```
print("%s %s %s" % ('Меня зовут', 'Андрей', 'Васечкин'))
```



При необходимости можно использовать модификатор длины, который выведет только указанное количество знаков после точки.

```
print("%.6s" % ('Хорошо в деревне летом!'))
```



Точно такой же результат можно получить следующим образом – вместо цифры указывая знак звёздочки. А сама цифра выносится в скобки.

```
print("%.5s" % (6, 'Хорошо в деревне летом!'))
```

```
1 x
```

```
Хорошо
```

Следующий флаг в виде пробела после оператора % с указанием числового значения заполняет пробелами справа на указанное количество символов.

```
print("%s" % ('Список сотрудников:\n'),
      "% 2s %s" % ('', 'Бухгалтерия:\n'),
      "% 10s %s" % ('', 'Петрова Алёна Васильевна\n'),
      "% 2s %s" % ('', 'Техотдел:\n'),
      "% 10s %s" % ('', 'Заболоцкий Виталий Романович'))
```

```
1 x
```

```
Список сотрудников:
```

```
    Бухгалтерия:
```

```
        Петрова Алёна Васильевна
```

```
    Техотдел:
```

```
        Заболоцкий Виталий Романович
```

Полностью аналогично будет, если использовать вместо пробела дефис. `%-2s %s" % ('', 'Бухгалтерия:\n')`

Флаг `+` заполняет пробелами свободное место слева. Причём подсчёт при использовании флагов всегда идёт с конца строки, из-за чего выравнивание строк разной длины таким способом очень непрактично.

```
print('%+16s' % 'Margin left')
print('%+10s' % 'Hello')
```

```
1 x
```

```
Margin left
```

```
Hello
```

Метод str.format()

Метод `format()` принимает произвольное количество аргументов, и выполняет их подстановку в указанных местах строки вместо фигурных скобок. Синтаксис схожий с оператором `%`, но данный метод гораздо гибче и функциональнее.

```
print('На дворе {}, на траве {}'.format('травы', 'дрова'))
```

1 x

На дворе травы, на траве дрова

Выводить строки можно по индексу в любом порядке. Отрицательные индексы НЕ поддерживаются. Если нужно вывести строки в порядке очереди, то номера индексов указывать необязательно.

```
res = '{0}\n{1}\n{2}'.format('BMW', 'AUDI', 'LADA')
print(res)
```

1 x

BMW
AUDI
LADA

```
res = '{2}\n{1}\n{0}'.format('BMW', 'AUDI', 'LADA')
print(res)
```

1 x

LADA
AUDI
BMW

Метод `format()` поддерживает распаковку строки.

```
res = '{2}{1}{0}{3}'.format(*'LADA')
print(res)
```

1 x

DALA

С помощью формат можно сделать конкатенацию строк.

```
res = '{2}{0}{1}'.format('Puper', 'Password', 'Super')
print(res)
```

1 x

SuperPuperPassword

Также, кроме позиционных, поддерживаются именованные аргументы.

```
res = 'Координаты по осям x,y: {x}, {y}'.format(x='20', y='150')
print(res)
```

```
1 x
```

```
Координаты по осям x,y: 20, 150
```

И смешанные. Позиционные аргументы должны следовать перед именованными.

```
res = 'Координаты {0} {x}, {y}'.format('по осям x,y:', x='20', y='150')
print(res)
```

```
1 x
```

```
Координаты по осям x,y: 20, 150
```

Аргументами могут быть данные любого типа.

```
res = 'Список запчастей: {} {}'.format({'фильтр':200, 'помпа':900,
    'ремень':400}, ('Итого 1500 рублей'))
print(res)
```

```
1 x
```

```
Список запчастей: {'фильтр': 200, 'помпа': 900, 'ремень': 400} Итого 1500 рублей
```

В данном примере объект разбивается на отдельные переменные, которые затем вставляются в строку.

```
shape = {'name': 'друг', 'theme': '"Форматирование"'}
print("Привет {name}, сегодня тема {theme}.".format(**shape))
```

```
1 x
```

```
Привет друг, сегодня тема "Форматирование".
```

Можно ещё выводить строки с заполнителем в виде спецсимволов.

```
res = '{0:~^14}'.format('Password')
print(res)
```

```
1 x
```

```
---Password---
```

```
res = '{0:~>14}'.format('Password')
print(res)
```

```
1 x
```

```
.....Password
```

Рассмотрим как это работает.

После индекса аргумента ставится двоеточие, после которого ставим символ, который будет являться заполнителем. Далее идут варианты как мы будем использовать заполнение:

Символ `^` указывает на заполнение с обеих сторон строки, количество символов берётся из цифры, которую мы указываем после `^`. Из этой цифры вычитается длина исходной строки и полученная цифра будет количеством символов, которые выведутся.

Символ `>` выведет заполнитель слева строки, а символ `<` справа. Если символ разделителя не указан, то по умолчанию в качестве разделителя будет использован пробел.

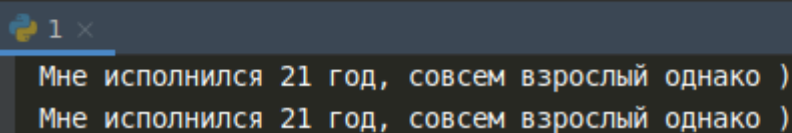
f-строки

С версии 3.6 в Python появились f-строки, расшифровывающиеся как «formatted string», то есть форматированная строка. **Этот способ форматирования считается наиболее современным и в ваших проектах стоит использовать именно его.**

Применение f-строк очень похоже на метод `format()`, отличается тем, что аргументы не нужно выносить в круглые скобки. Посмотрите на примере отличие:

```
int = 21
string = 'взрослый'

# Метод формат
print('Мне исполнился {} год, совсем {} однако ').format(int, string))
# f-строки
print(f'Мне исполнился {int} год, совсем {string} однако ')
```

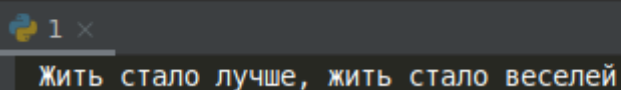


```
1 x
Мне исполнился 21 год, совсем взрослый однако )
Мне исполнился 21 год, совсем взрослый однако )
```

Как можно заметить, запись, выполненная с использованием f-strings короче и удобнее, так как аргументы визуально ставятся именно в те места, где они и будут. Кроме того, f-строки ещё и быстрее выполняются, а значит в проекте требовательном к скорости выполнения, являются предпочтительным методом форматирования.

К значениям в списке по индексу мы теперь обращаемся прямо в строках.

```
how = ["бедней", "веселей", "лучше", "хуже"]
print(f"Жить стало {how[2]}, жить стало {how[1]}")
```



```
1 x
Жить стало лучше, жить стало веселей
```

Вызов метода объекта.

```
name = "pentester"
print(f"{name.upper()}")
```

1 x

PENTESTER

Обращение к словарю по ключу, многострочные f-строки.
Обратите внимание! В str.format ключ из словаря мы брали без кавычек, а в f-strings кавычки обязательны!

```
salary = {"boss": "150к", "engineer": "80к", "janitor": "20к"}
print(f"Получил зарплату {salary['janitor']}? Не валяй дурака!\n"
      f"Хочешь {salary['engineer']}? Подучись ещё пока )\n"
      f"Шляпу белую одел, {salary['boss']} не предел!")
```

1 x

Получил зарплату 20к? Не валяй дурака!
 Хочешь 80к? Подучись ещё пока)
 Шляпу белую одел, 150к не предел!

Использование встроенных функций.

```
print(f"{len('word')}")
```

1 x

4

Ещё одна удобное свойство f-строк позволяет нам вывести на печать название переменной и её содержимое, указав после имени переменной в фигурных скобках символ равенства. Пользуйтесь этим приёмом во время отладки ваших приложений

```
1 my_books = {'Даниэль Дефо': 'Робинзон Крузо',
2             'Жюль Верн': 'Таинственный остров'}
3 for key, val in my_books.items():
4     print(f'Имя автора: {key=} \tНазвание книги {val=}')
5
```

'Жюль Верн'

Run: x

Имя автора: key='Даниэль Дефо' Название книги val='Робинзон Крузо'
 Имя автора: key='Жюль Верн' Название книги val='Таинственный остров'

Более сложная конструкция – вызов пользовательской функции прямо в f-строках.

```
def f2():  
    choice = ['flash', 'lambada']  
    return f"{choice[1]}"  
  
print(f"Длина слова {f2()} {len(f2())} символов")
```

1 ×
Длина слова lambada 7 символов

Заполнение пустого пространства работает аналогично методу формат.

```
title = "CODEBY"  
print(f"{title:^12}")  
print(f"{title:<12}")  
print(f"{title:>12}")
```

1 ×
---CODEBY---
CODEBY-----
-----CODEBY

Форматирование чисел

Для оператора % целые числа определяются как %d.

```
num = 26
print("Мне нравится число %d" % num)
```

1 x

Мне нравится число 26

Числа с плавающей точкой обозначаются %f. По-умолчанию выводится 6 знаков после точки. Мы можем управлять желаемой точностью, задав нужную цифру после точки, которая добавляется за оператором %.

```
num = 36.6
print("Нормальная температура %f" % num)
print("Нормальная температура %.1f" % num)
```

1 x

Нормальная температура 36.600000
Нормальная температура 36.6

Выставив значение ноль, выведется число без плавающей точки.

```
num = 40
print("Сегодня жарко, +%.0f в тени" % num)
```

1 x

Сегодня жарко, +40 в тени

Метод формат похож, только принадлежность к типу данных и точность знаков выставляется после двоеточия.

```
num = 1000
print('{:d}'.format(num))
print('{:f}'.format(1.3333333))
print('{:.2f}'.format(1.3333333))
```

1 x

1000
1.333333
1.33

Ещё один числовой параметр, добавленный после двоеточия, позволяет установить минимальную ширину форматируемого значения с выравниванием по правому краю.

```
num = 1000
print('{:8d}'.format(num))
print('{:12f}'.format(1.33333333))
print('{:8.2f}'.format(1.33333333))
```

1 x

```
1000
1.333333
1.33
```

В качестве заполнителя пустого пространства можно использовать только нули.

```
print("{:<06d}".format(17))
print("{:>06d}".format(17))
print("{:^06d}".format(17))
```

1 x

```
170000
000017
001700
```

Числа также можно распаковать из списка или кортежа и вывести по индексу.

```
nums = [43, 44, 100.1, 6, 502, -10]
print("{2}, {5}, {0}".format(*nums))
```

1 x

```
100.1, -10, 43
```

Для f-строк всё выглядит практически также – любые манипуляции ставятся после двоеточия.

```
hours = 1
minutes = 59
seconds = 3
print(f'{hours:02d}:{minutes:02d}:{seconds:02d}')
```

1 x

```
01:59:03
```

Стандартные математические операции.

```
a = 52
b = 59
seconds = 3
print(f"{a*b}\n{a+b}\n{a/b:.3}")
```

```
1 x
3068
111
0.881
```

Если число больше 999, то можно в качестве разделителя добавить запятую.

```
num = 1000
print(f'{num:f}')
print(f'{num:.2f}')
print(f'{num:,.2f}')
```

```
1 x
1000.000000
1000.00
1,000.00
```